Proyecto Intermodular PROMETEO: Gestión de Bases de Datos

Curso: 25-26 | **Ciclo:** 1º ASIR

Autor: Guillermo Duran Cortes

Módulo: Gestión de Bases de Datos (0372)

En este documento voy a detallar el diseño, creación y administración de la base de datos central para **BugiSoft**, un estudio ficticio especializado en el desarrollo de videojuegos AAA y servicios *Live Service*.

El objetivo de esta base de datos es gestionar la información crítica que permite el funcionamiento diario del estudio: organizar a los empleados, llevar el control de los proyectos en desarrollo y gestionar el inventario del hardware que soporta toda la infraestructura de la empresa.

1. Análisis de los datos del sistema

A continuación explico qué datos guardamos, por qué son necesarios y cómo se relacionan entre sí:

A. Empleados y departamentos

- **Qué guardamos:** El identificador único de cada empleado, su nombre completo, su rol y el departamento al que está asignado.
- **Por qué hace falta:** Una empresa de desarrollo necesita organizar a su personal en equipos de trabajo claros.
- **Cómo se relaciona con el sistema:** Nos permite vincular a cada empleado con los proyectos en los que trabaja y asignarles el hardware específico que necesitan según su departamento.

B. Proyectos

- **Qué guardamos:** El ID del proyecto, el título del videojuego, la fase actual de desarrollo y la fecha estimada de lanzamiento.

- **Por qué hace falta:** La empresa necesita saber en todo momento en qué estado se encuentra cada título y organizar los recursos para cumplir con las fechas de entrega.
- **Cómo se relaciona con el sistema:** Los proyectos están conectados directamente con los departamentos y empleados. De esta forma, el sistema puede consultar fácilmente qué desarrolladores o testers están asignados a un videojuego específico.

C. Inventario de hardware y equipos

- **Qué guardamos:** El identificador del equipo, el tipo de máquina (PC de desarrollo, servidor de datos, servidor web), la dirección IP asignada y el departamento que lo utiliza.
- **Por qué hace falta:** Necesitamos tener un control riguroso de la infraestructura física y lógica de la empresa para poder hacer mantenimientos, resolver incidencias o gestionar ampliaciones.
- **Cómo se relaciona con el sistema:** Conecta a los empleados con las herramientas que usan.

D. Control de calidad (tickets / QA)

- **Qué guardamos:** El registro de los *bugs* o errores encontrados, su estado actual, a qué videojuego pertenecen y qué empleado lo reportó.
- **Por qué hace falta:** Para asegurar que los juegos salen al mercado sin fallos y llevar un seguimiento del trabajo de los *testers*.
- **Cómo se relaciona con el sistema:** Conecta a los empleados de ese departamento con los proyectos que están testeando en tiempo real.

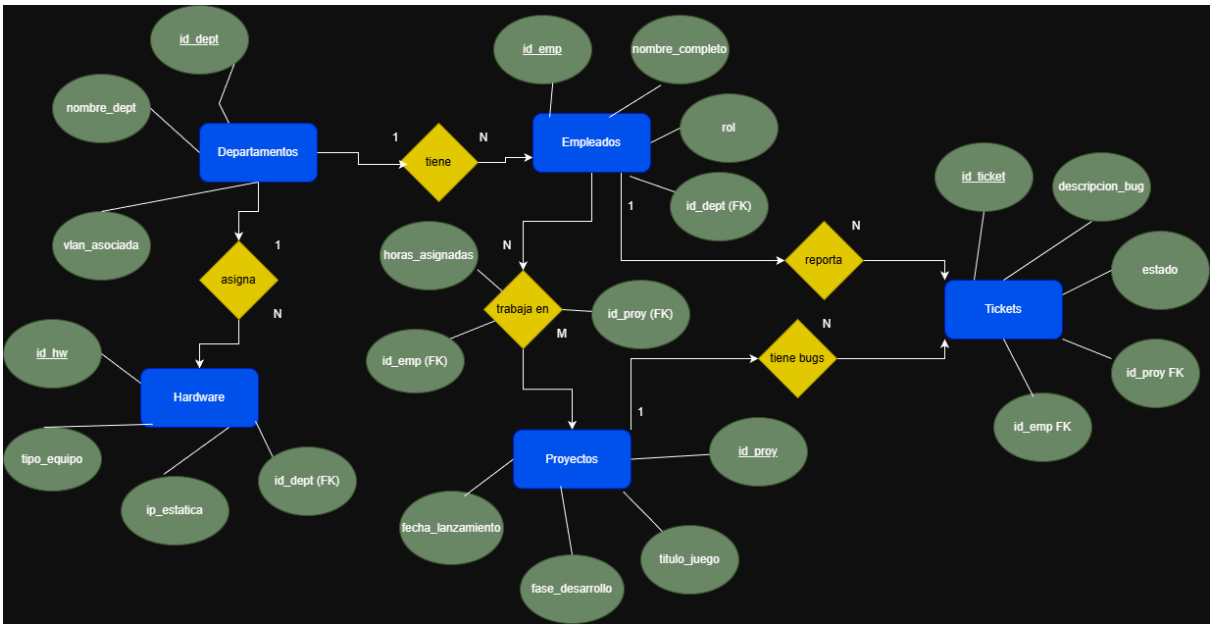
---

2. Diseño de la base de datos

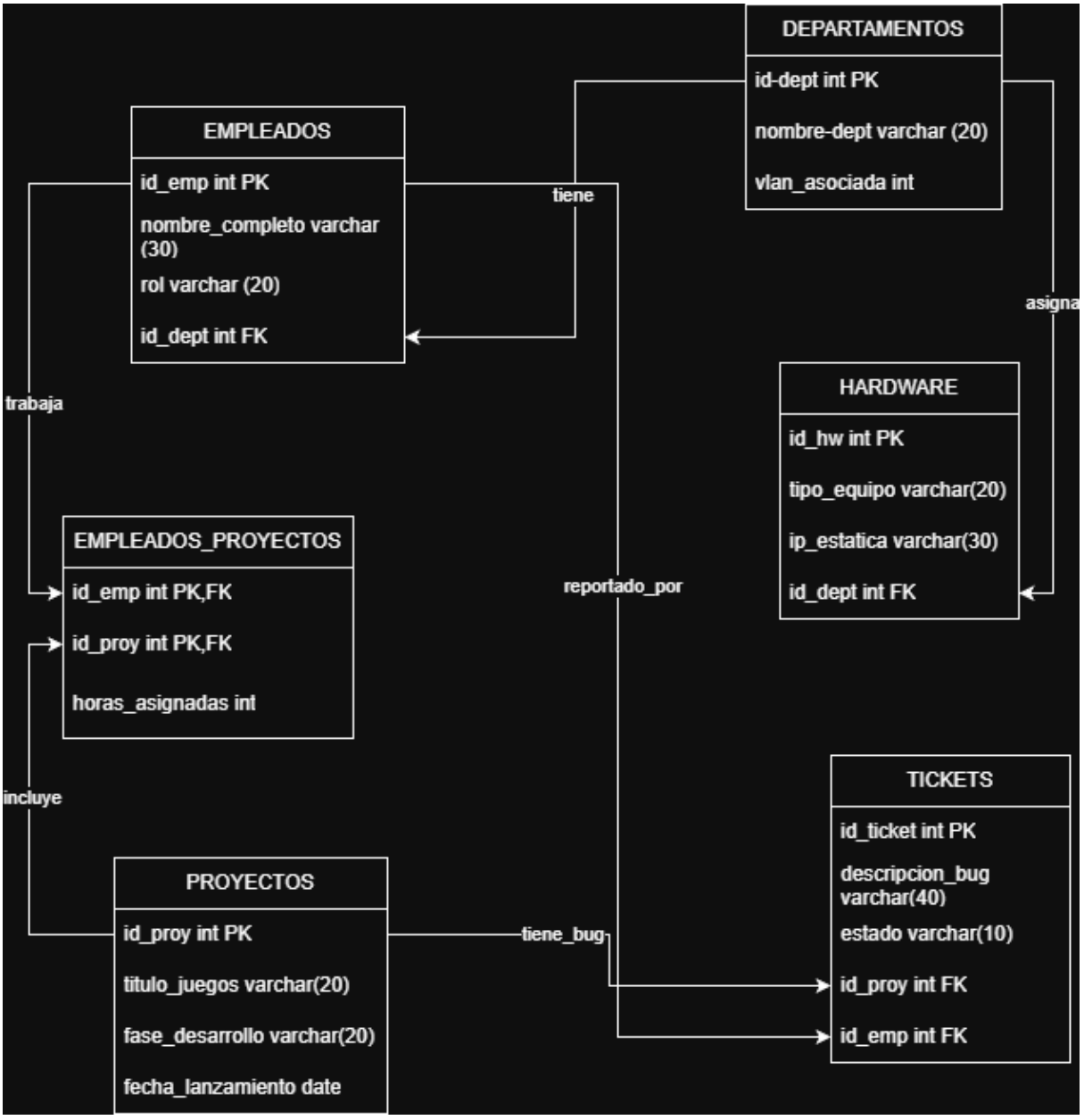
Una vez que tenemos claro qué información necesita BugiSoft, el siguiente paso es traducir esas ideas a una estructura técnica real. Para organizar los datos de forma eficiente y evitar que la información se duplique o se pierda, he diseñado una base de datos compuesta por 5 tablas.

He decidido incluir una tabla intermedia (Empleados_Proyectos) porque en un estudio de videojuegos es muy común que un mismo desarrollador o tester trabaje en varios proyectos a la vez y, obviamente, un proyecto tiene a muchos empleados asignados.

A continuación detallo el modelo E/R:



Y aquí presento el modelo relacional:



3. Creación de la base de datos (tablas)

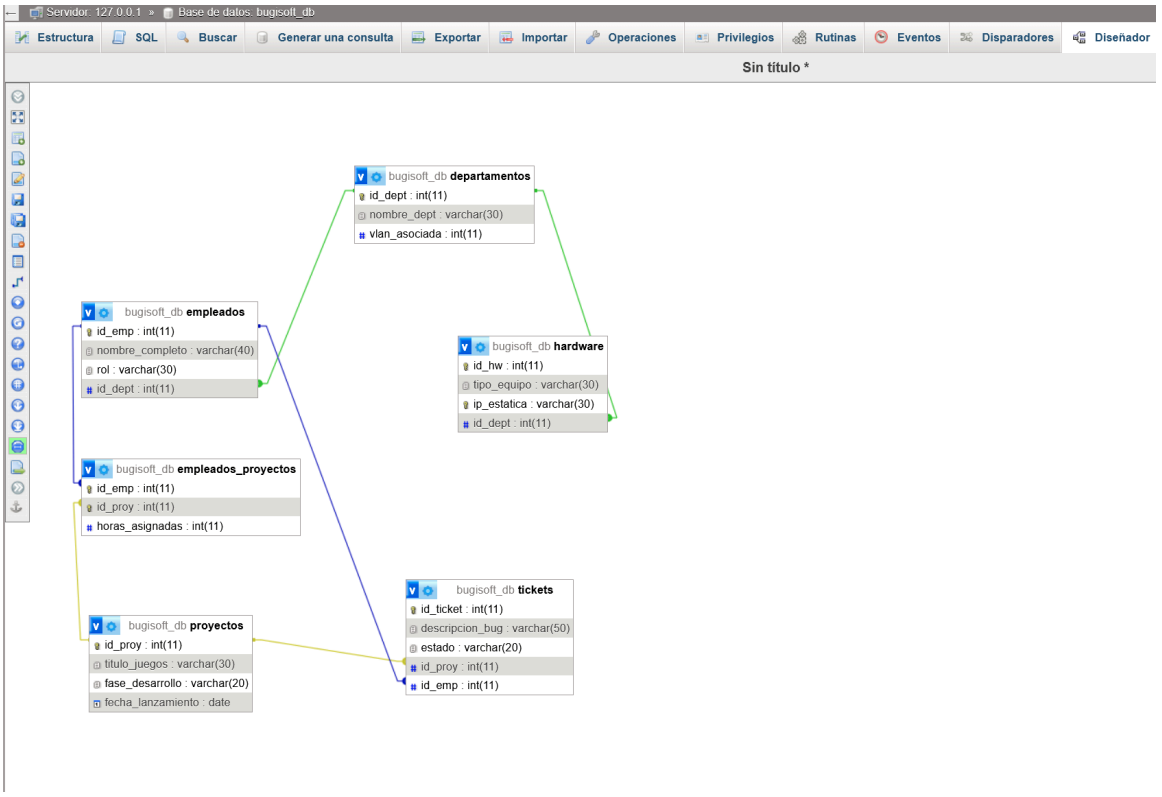
Una vez definido el modelo relacional, se ha procedido a su implementación en el gestor **MariaDB (XAMPP)**. Obviamente, todo el código está representado en el archivo base_datos.sql, bien ordenado y estructurado.

Como muestra de la estructura, presento la creación de la tabla Empleados:

```
CREATE TABLE Empleados (
    id_emp INT AUTO_INCREMENT PRIMARY KEY,
    nombre_completo VARCHAR(30) NOT NULL,
    rol VARCHAR(20),
    id_dept INT,
```

```
    CONSTRAINT fk_empleado_dept FOREIGN KEY (id_dept) REFER
ENCES Departamentos(id_dept)
);
```

Y aquí viene una representación visual de toda la base de datos en MyPhpAdmin:



4. Inserción de datos y población de la BD

Tras la creación de la estructura, he metido en la base de datos un conjunto de registros realistas que simulan la actividad diaria de BugiSoft, incluyendo perfiles personalizados para el equipo docente, como Luz María en el rol de jefa de QA y Sergio Gallardo en Formación. Al vincular cada empleado y equipo con su departamento correspondiente, la base de datos queda lista para realizar pruebas de integridad y consultas de administración que reflejan fielmente el funcionamiento lógico de la red corporativa.

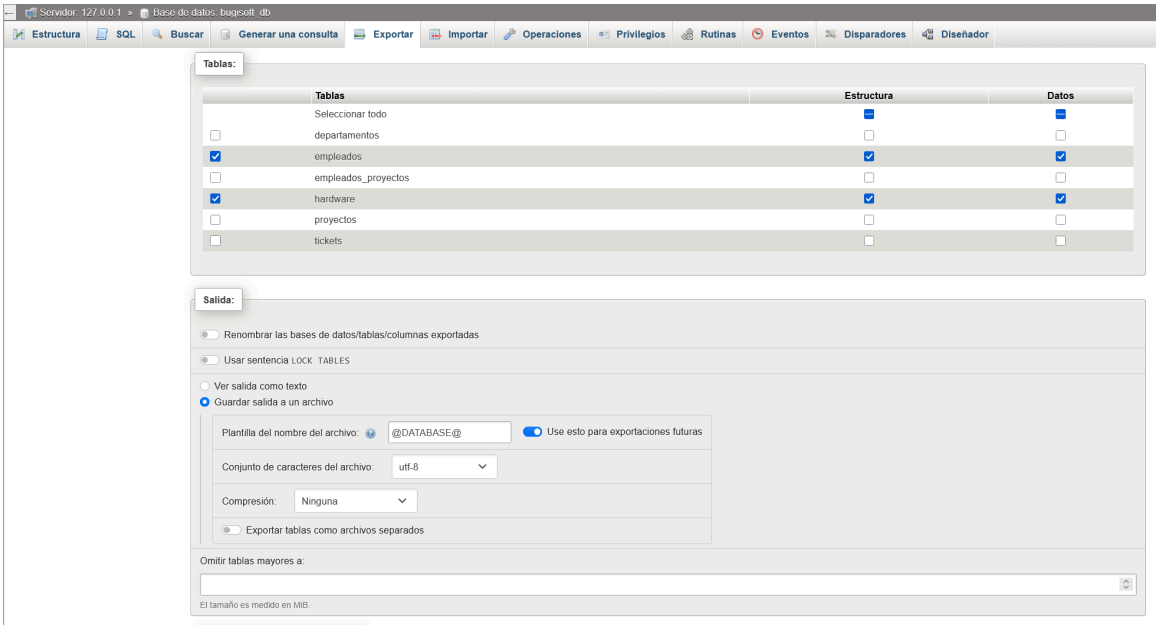
El listado completo de sentencias `INSERT` para todas las tablas se adjunta en el archivo independiente `insercion_datos.sql`

5. Tareas de administración básica

Para garantizar que la infraestructura de BugiSoft sea segura, se han definido los siguientes protocolos de gestión técnica:

Copias de seguridad: Se realizan exportaciones periódicas en formato SQL mediante la herramienta nativa de phpMyAdmin. Estos archivos de respaldo permiten reconstruir la base de datos completa en caso de fallo crítico en el servidor principal (`192.168.60.2`).

Exportación de datos: Se han validado procesos de migración de datos para auditorías externas, asegurando que la información de hardware y empleados mantenga su consistencia referencial durante el traspaso entre entornos de desarrollo y producción.



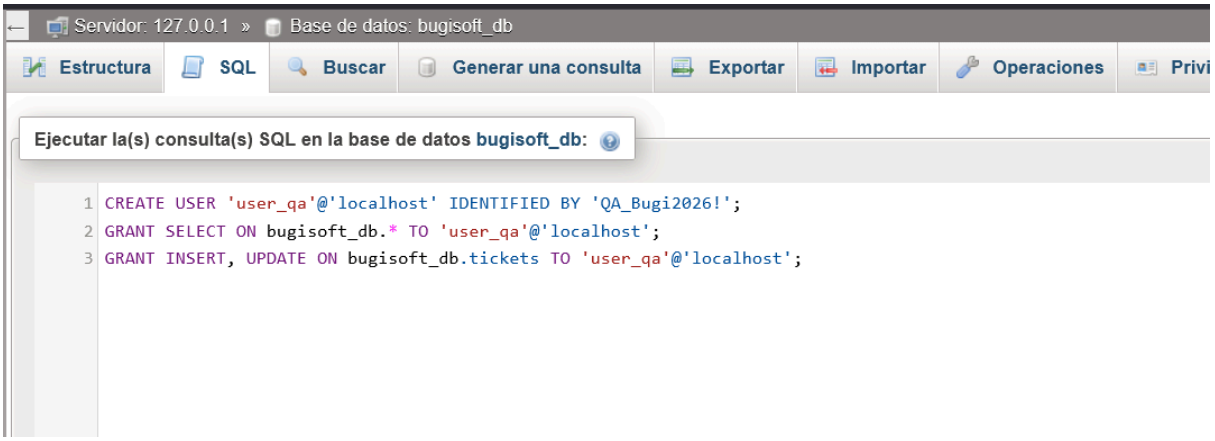
Gestión básica de usuarios: El acceso administrativo se restringe al perfil de **Experto Redes**. Se han diseñado roles específicos con permisos limitados (usando sentencias `GRANT`), permitiendo que el personal de **SOPORTE_QA** pueda gestionar tickets sin riesgo de alterar la estructura de las tablas del sistema.

Creo el administrador con acceso total a todo (yo):

```
CREATE USER 'admin_redes'@'localhost' IDENTIFIED BY 'AdminB
ugi2026!';
GRANT ALL PRIVILEGES ON bugisoft_db.* TO 'admin_redes'@'loc
alhost'
```

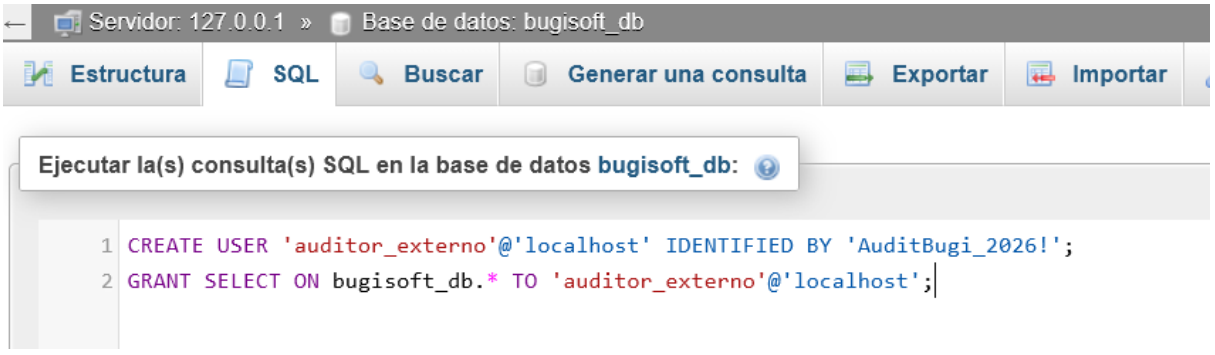
Creo además un usuario que pertenece al área de Soporte, en el que se le darán permisos para añadir y actualizar las tablas, pero sin control total del resto de la base de datos:

```
CREATE USER 'user_qa'@'localhost' IDENTIFIED BY 'QA_Bugi202
6!';
GRANT SELECT ON bugisoft_db.* TO 'user_qa'@'localhost';
GRANT INSERT, UPDATE ON bugisoft_db.tickets TO 'user_qa'@'l
ocalhost';
```

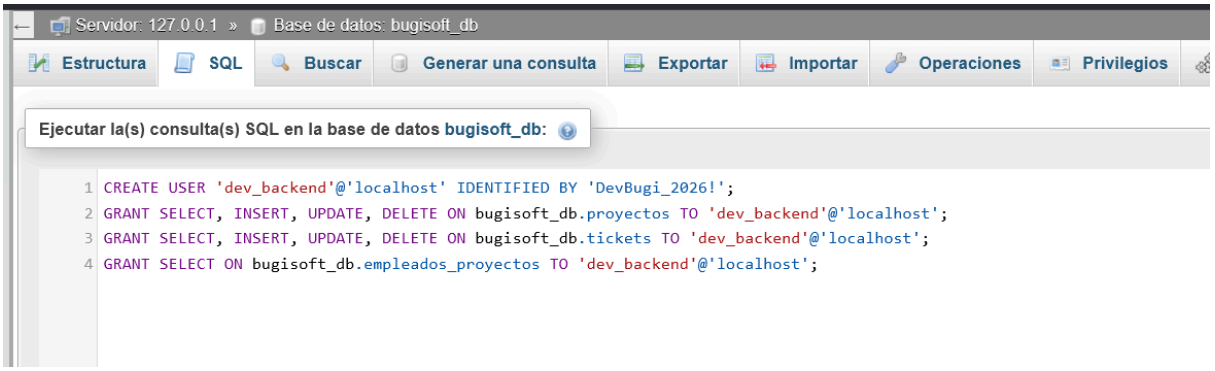


Un auditor externo o el jefe del estudio necesita ver absolutamente todo para sacar informes, pero por seguridad no puede modificar ni borrar nada.

```
CREATE USER 'auditor_externo'@'localhost' IDENTIFIED BY 'Au
ditBugi_2026!';
GRANT SELECT ON bugisoft_db.* TO 'auditor_externo'@'localho
st';
```



Los programadores necesitan leer y modificar datos sobre los juegos que están haciendo y los bugs que tienen que arreglar, pero no deben poder tocar el inventario de Hardware ni los salarios de los empleados.

```
CREATE USER 'dev_backend'@'localhost' IDENTIFIED BY 'DevBug
i_2026!';
GRANT SELECT, INSERT, UPDATE, DELETE ON bugisoft_db.proyect
```

```
os TO 'dev_backend'@'localhost';
GRANT SELECT, INSERT, UPDATE, DELETE ON bugisoft_db.tickets
TO 'dev_backend'@'localhost';
GRANT SELECT ON bugisoft_db.empleados_proyectos TO 'dev_bac
kend'@'localhost';
```